

Thiết kế thí nghiệm, ablation và phân tích lỗi (Experiment Design, Ablation, Error Analysis)

PHỤ THUỘC B/07-chia-du-lieu-va-danh-gia (chia dữ liệu, chọn metric — chương này giả định bạn đã có một giao thức đánh giá đúng); D/22-tai-lap-va-mlops (config, tracking, nguồn phi tắt định); E/23-doc-paper là mặt đọc của cùng kỹ năng này.

TÓM TẮT

Chương này là về cách *sinh ra bằng chứng*: thiết kế một so sánh có nghĩa, cắt bỏ thành phần đúng cách, biết bao nhiêu lần chạy là đủ, và đọc lỗi để quyết định đầu tư tiếp vào đâu. Đọc xong bạn tính được cỡ mẫu cần thiết từ sản nhiễu, chọn được giữa leave-one-out và oracle experiment cho đúng câu hỏi mình đang hỏi, và viết được một bảng ablation mà người hoài nghi nhất cũng dùng được. Với nền SLAM sẵn có, phần bạn còn thiếu không phải kỹ thuật lặp lại nhiều lần mà là ba thứ đặc thù deep learning: phi tắt định không sửa được, bất đối xứng ngân sách tune, và rò rỉ tập test tích lũy qua nhiều tháng. Nếu chỉ nhớ một điều: đo sản nhiễu *trước*, nói về cải tiến *sau* — và một kết luận âm ghi kèm điều kiện thí nghiệm vẫn là một kết quả.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ablation** — thí nghiệm bỏ đi hoặc thay thế đúng *một* thành phần của hệ thống rồi đo lại, để biết thành phần đó đóng góp bao nhiêu.
- **baseline** — phương pháp đối chứng mà kết quả mới phải vượt qua; baseline chưa được tinh chỉnh từ tế làm mọi so sánh sau đó vô nghĩa.
- **seed và phương sai giữa các lần chạy** — seed là số khởi tạo bộ sinh ngẫu nhiên; chạy lại cùng cấu hình với seed khác cho kết quả khác, và độ tản mát của các kết quả đó là phương sai giữa các lần chạy — phần bạn đã quen từ harness SLAM, ở đây chỉ đổi nguồn gây ra nó.
- **độ lệch chuẩn σ** — thước đo độ tản mát của một tập số quanh trung bình của chúng. Trong chương này σ luôn là độ lệch chuẩn *giữa các seed* của cùng một cấu hình, tức là “sản nhiễu”.
- **khoảng tin cậy** — một khoảng số kèm mức tin cậy (thường 95%), tính từ dữ liệu, sao cho nếu lặp lại toàn bộ quy trình nhiều lần thì khoảng ấy chứa giá trị thật trong đúng tỉ lệ đó. Nó trả lời “tôi biết con số này chính xác tới đâu”, không trả lời “giả thuyết của tôi đúng hay sai”.
- **p-value** — xác suất quan sát được chênh lệch ít nhất bằng chênh lệch đã thấy, *nếu* giả thuyết “không có khác biệt” là đúng. Nó *không* phải xác suất giả thuyết của bạn đúng, và một p-value nhỏ trên tập test rất lớn có thể ứng với một chênh lệch nhỏ tới mức vô nghĩa về mặt triển khai.
- **cỡ hiệu ứng (effect size)** — độ lớn của chênh lệch, đo bằng đơn vị của bài toán (0,8 mAP) hoặc chuẩn hoá theo σ . Đây mới là đại lượng quyết định “có đáng triển khai không”.
- **oracle experiment** — thay một khối trong hệ thống bằng đáp án đúng, tức **chân trị** (*ground truth*), để đo *trần* đóng góp của khối đó. Nếu khối hoàn hảo cũng chỉ nâng hệ thống 2% thì mọi công sức tối ưu nó đều bị chặn trên bởi 2%.
- **mAP** — metric tổng hợp thông dụng cho phát hiện vật thể; ở đây chỉ cần hiểu là “điểm số càng cao càng tốt”, tính theo điểm phần trăm.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn thử hai mươi cấu hình rồi lấy cái tốt nhất và thấy nó hơn baseline 1,1 điểm. Vì sao con số đó có thể hoàn toàn là nhiễu — và bạn tính ra điều đó bằng phép tính nào, trong ba mươi giây?
- Q2. Quy tắc “chạy ba lần” mà bạn đã dùng cho harness SLAM là đúng, nhưng ba là bao nhiêu phần trăm của cái cần thiết? Với sàn nhiễu 0,6 điểm, ba lần chạy bỏ sót bao nhiêu phần số cải tiến có thật?
- Q3. Bạn bỏ một module và mAP giảm 1,5. Kể ra bốn cách giải thích khác ngoài “module đó quan trọng” — và cách nào trong số đó luôn dẫn kết luận về cùng một hướng?
- Q4. Nếu một detector hoàn hảo cũng chỉ nâng hệ thống tổng thể 2%, bạn còn lý do gì để tối ưu detector — và làm sao biết được con số 2% ấy *trước* khi bỏ ba tháng?
- Q5. Vì sao một kết luận NO-GO không ghi điều kiện thí nghiệm lại có hại hơn là không ghi gì cả?

1.1 Đặt vấn đề

Chương 7 nói về *giao thức đánh giá*: chia dữ liệu thế nào, đo bằng metric gì. Chương này nói về một việc khác hẳn — *sinh ra bằng chứng*: làm thế nào để một so sánh có nghĩa, và làm thế nào để biết nên đầu tư công sức tiếp theo vào đâu. Đây là kỹ năng phân biệt người làm nghiên cứu với người chạy script, và nó gần như không được dạy ở đâu cả; hầu hết người làm nghề học nó bằng cách tự trả giá.

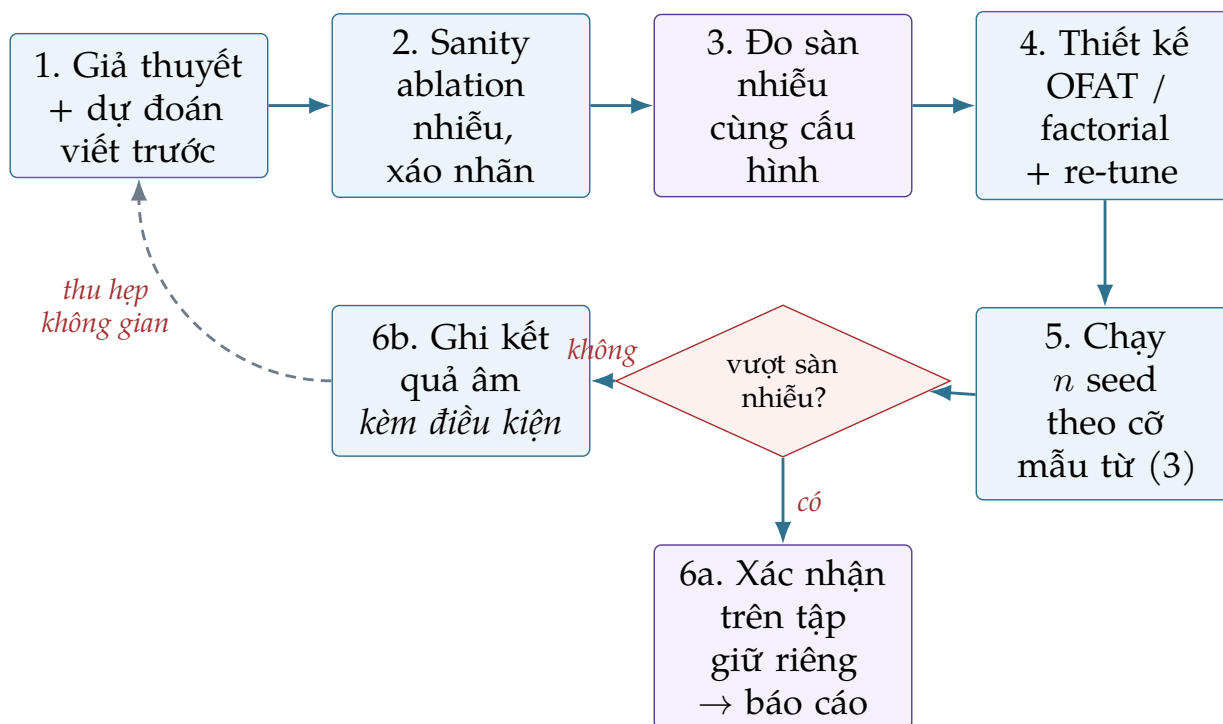
Bạn đã trả một phần cái giá đó rồi, ở phía SLAM. Khi dựng một harness chạy lặp trên EuRoC, bạn phát hiện chính hệ thống của mình là phi tất định: hệ thống có hai nguồn phi tất định cùng lúc.

- **Thứ tự duyệt container** — các container đánh chỉ mục bằng con trỏ (*pointer-ordered containers*) đổi thứ tự theo cách cấp phát bộ nhớ.
- **Tranh chấp luồng** (*race condition*) — giữa luồng tracking và luồng local mapping, khiến một cờ trạng thái đọc ra khác nhau giữa các lần chạy.

Hệ quả giống nhau ở cả hai nguồn: chênh lệch một inlier lan thành một quyết định keyframe khác, rồi thành một quỹ đạo khác. Từ đó bạn đặt quy tắc: mọi cấu hình chạy ít nhất ba lần với ba *seed* khác nhau. Bạn cũng đã tự viết những loạt ablation nhiều cấu hình và rút ra vài kết luận NO-GO, tức là đã biết bằng kinh nghiệm rằng phần lớn ý tưởng nghe hay đều chết, và rằng biết chúng chết là một kết quả.

Chương này vì thế không giảng lại nhóm C từ đầu. Nó hệ thống hoá những gì bạn đã biết thành một khung có tên gọi, để bạn áp được sang bài toán mới mà không phải phát hiện lại, và chỉ ra ba chỗ mà môi trường deep learning *khác*. Thứ nhất, nguồn phi tất định ở đây không sửa được — không có bug nào để tìm — nên mục tiêu phải chuyển từ “tái lập từng chữ số” sang “tái lập phân phối kết quả”. Thứ hai, không gian siêu tham số lớn hơn nhiều, nên bất đối xứng ngân sách tune giữa phương pháp mới và baseline trở thành nguồn kết luận sai lớn nhất, lớn hơn cả nhiễu seed. Thứ ba, tập test bị dùng lại hàng trăm lần trong nhiều tháng. Nó còn có thể đã nằm sẵn trong dữ liệu pretraining của mô hình nền bạn dùng. Đó là hai dạng rò rỉ mà một pipeline SLAM chạy trên EuRoC cũng mắc, chỉ là ít khi được gọi tên.

Hình 1.1 tóm cả chương thành một quy trình. Đọc nó trước rồi mới đọc năm nhóm bên dưới sẽ nhanh hơn nhiều, vì mỗi nhóm chỉ là một trạm trên đúng đường đi này.



Hình 1.1. Từ giả thuyết tới kết luận. Ba chỗ hay bị bỏ qua nhất được tô đậm hoặc rẽ nhánh. Bước 3: đo sản nhiều *trước* khi so sánh. Bước 5: tính cỡ mẫu từ chính sản nhiều đó. Bước 6b: coi nhánh “không vượt sản” là một kết quả phải ghi lại, không phải một lần chạy bỏ đi; mũi tên đứt là chỗ nó trả giá trị về cho vòng sau.

TRỰC GIÁC

Bỏ một module ra khỏi hệ thống rồi giữ nguyên learning rate cũ giống tháo giảm chấn khỏi hệ treo rồi vẫn chạy thử ở đúng tốc độ và đúng áp suất lốp cũ. Con số bạn đo được không phải “xe không có giảm chấn” mà là “xe không có giảm chấn *cộng* với một bộ tham số vận hành đã trở nên sai cho nó”. Sai lệch này luôn cùng chiều — nó thổi phồng tầm quan trọng của thành phần bị bỏ — nên nó không tự triệt tiêu khi bạn lặp lại nhiều lần.

1.2 Nhóm A — thiết kế so sánh (Comparison Design)

A. Ba loại câu hỏi, ba thiết kế

Lỗi phổ biến nhất trong thiết kế thí nghiệm không phải lỗi thống kê mà là lỗi phân loại: dùng một thiết kế để trả lời câu hỏi mà nó không trả lời được. Có đúng ba loại câu hỏi và chúng cần ba thiết kế khác nhau. “Cái này có hoạt động không” là so sánh với baseline, và thiết kế của nó đòi hỏi kiểm soát chặt mọi biến gây nhiễu cùng nhiều seed. “Vì sao nó hoạt động” là ablation, và thiết kế của nó đòi hỏi mỗi lần chỉ đổi một thứ, kèm re-tune. “Nó hoạt động tới đâu” là sensitivity và scaling. Thiết kế của nó cần nhiều điểm dọc một trục: kích thước dữ liệu, kích thước mô hình, hoặc mức nhiễu. Bù lại nó chấp nhận ít seed mỗi điểm, vì cái ta đọc là *xu hướng* chứ không phải một hiệu số. Lấy một bảng ablation ra để chứng minh phương pháp hoạt động, hoặc lấy một so sánh baseline ra để giải thích cơ chế, là hai cách trộn lẫn hay gặp nhất và cả hai đều cho kết luận không đứng vững.

B. Ba ràng buộc phải thoả trước khi con số có nghĩa

Trên nền đó, ba ràng buộc phải thoả trước khi bất kỳ con số nào có nghĩa. **Baseline mạnh là nghĩa vụ, không phải phép lịch sự:** so với baseline bị tune kém là cách tự lừa mình phổ biến nhất, và nó phổ biến vì rẻ. **Kiểm soát biến gây nhiễu:** cùng dữ liệu, cùng augmentation, cùng lịch huấn luyện, cùng số seed. Nếu phương pháp mới đắt hơn 1,4 lần mỗi bước mà vẫn chạy cùng số epoch, nó đã được tặng thêm 40% tính toán. Riêng lượng tính toán thêm đó đủ tạo khác biệt đo được với các công thức huấn luyện hiện đại [1]. **Ngân sách tune công bằng:** cơ chế ở đây thuần xác suất — giá trị tốt nhất quan sát được là cực đại của một mẫu, và kỳ vọng của cực đại tăng theo cỡ mẫu [2], nên một cải tiến 1% rút ra từ 50 lần thử so với baseline được thử 5 lần không phải là cải tiến mà là hiệu ứng của cỡ mẫu.

→ **hệ quả của E/23** — đây chính là dấu hiệu “ngân sách tính chính bất đối xứng” mà bạn dùng để đọc bảng kết quả của người khác

C. Một-biến-một-lần hay giai thừa

Còn một lựa chọn thiết kế nữa hay bị bỏ qua: một-biến-một-lần (OFAT) hay giai thừa (factorial). OFAT rẻ và trực giác, nhưng mù hoàn toàn với tương tác — nếu hai thành phần cộng hưởng hoặc triệt tiêu nhau, OFAT không thể phát hiện. Với ba thành phần, một thiết kế giai thừa đầy đủ có $2^3 = 8$ cấu hình, nhân ba seed là 24 lần chạy; OFAT chỉ cần cấu hình gốc cộng ba lần bỏ, tức 4 cấu hình và 12 lần chạy. Gấp đôi chi phí để đổi lấy toàn bộ tương tác bậc hai thường là món hời. Khi số thành phần lớn hơn, thiết kế giai thừa rút gọn cho phép trả ít hơn nhiều so với 2^k mà vẫn tách được các tương tác đáng kể.

I 1.3 Nhóm B — kỹ thuật ablation (Ablation Techniques)

A. Hai chiều bỏ thành phần, hai câu hỏi khác nhau

Có hai chiều bỏ thành phần, và chúng trả lời hai câu hỏi khác nhau. *leave-one-out* bỏ từng thành phần khỏi hệ thống đầy đủ; *add-one-in* thêm từng thành phần vào baseline trần. Khi các thành phần độc lập, hai chiều cho cùng con số. Khi có tương tác, chúng tách ra, và *chính khoảng cách giữa chúng là thông tin*. Ví dụ nhỏ nhất: giả sử thêm riêng A được +1,0, thêm riêng B được +1,0, nhưng A và B cùng lúc chỉ được +1,2. Khi đó *leave-one-out* báo “bỏ A mất 0,2” còn *add-one-in* báo “thêm A được 1,0”. Cả hai đều đúng; chúng chỉ trả lời hai câu khác nhau — “A còn đáng giữ trong hệ thống này không” và “A có tự nó tạo ra giá trị không”. Chỉ báo cáo một chiều rồi phát biểu như thể đó là “đóng góp của A” là chỗ mập mờ hay gập nhất trong các bảng ablation đã công bố.

B. Ba loại đối tượng, và luật re-tune

Ablation có ba loại theo đối tượng bị bỏ: kiến trúc, dữ liệu, và công thức huấn luyện. Hai loại sau gần như luôn bị bỏ qua, dù chúng gần như luôn giải thích phần lớn chênh lệch. Đây chính là mặt trong của kết luận ở chương 23 về đóng góp thật so với recipe: nếu bạn không ablate recipe, bạn không có cách nào biết đóng góp của mình có thật không.

Ràng buộc quan trọng nhất của cả nhóm là **ablation phải re-tune**, vì lý do ở hộp trực giác: bỏ một thành phần có tác dụng ổn định hoá thì learning rate tối ưu của phần còn lại thường nhỏ đi, nên giữ nguyên learning rate cũ vừa làm hệ thống tệ hơn vừa thổi phồng tầm quan trọng của thành phần vừa bỏ. Thoả hiệp thực dụng là re-tune *ít nhất* learning rate trên một lưới thô ba điểm và ghi rõ rằng chỉ có learning rate được re-tune.

C. Sanity ablation chạy trước

Trước mọi ablation tinh vi, hãy chạy nhóm **sanity ablation**: thay đầu vào bằng nhiễu, xáo trộn nhãn, cắt bỏ một modality, thay một module bằng hằng số. Nếu điểm không giảm thì hoặc có rò rỉ, hoặc thành phần đó vô dụng, và cả hai kết luận đều dừng cuộc điều tra lại tại chỗ. Nhóm này rẻ và có sức phá hoại giả thuyết cao nhất, nên chạy trước là đúng thứ tự kinh tế. Một lưu ý về diễn giải: mạng đủ lớn có thể học thuộc cả một tập nhãn ngẫu nhiên, nên “train loss vẫn giảm khi xáo nhãn” không phải dấu hiệu rò rỉ — chỉ có *điểm trên tập đánh giá* không giảm mới là [3].

D. Oracle experiment và ablation theo quy mô

Công cụ mạnh nhất để quyết định đầu tư lại là loại ablation ngược: **oracle** hay **skyline**, tức thay một khối bằng chân trị và đo trần đóng góp của khối đó. Nếu một detector hoàn hảo chỉ nâng hệ thống tổng thể lên 2%, thì mọi công sức tối ưu detector đều bị chặn trên bởi 2% và bạn nên đi chỗ khác. Bạn đã dùng đúng công cụ này ở phía SLAM khi thay tầng phát hiện vòng lặp bằng một oracle lấy từ chân trị để tách “không phát hiện được vòng lặp” khỏi “phát hiện được nhưng không dùng được” — và kết luận thu được đã định hướng lại toàn bộ hướng làm việc, đó là điều một ablation thông thường không làm được. Cuối cùng, mọi kết luận ablation đều gắn với quy mô: kết luận rút ra trên 1% dữ liệu thường đảo ngược ở 100%, vì các thành phần có tác dụng chống quá khớp mất ý nghĩa khi dữ liệu đủ nhiều. Luôn ghi ablation chạy ở quy mô nào.

→ **trường hợp riêng của B/03** — oracle experiment là cách trả lời bằng số cho câu hỏi “độ khó đã bị đẩy đi đâu”: nó xoá hẳn độ khó của một khối rồi đo phần còn lại

Bảng 1.1. Năm kiểu ablation, mỗi kiểu trả lời một câu hỏi khác nhau. Cột cuối là cột đáng giá nhất: nó cho biết khi nào con số thu được không có nghĩa như ta tưởng.

Kiểu	Trả lời câu hỏi	Giả định	Hỏng khi nào
Leave-one-out	Thành phần này còn đáng giữ không	Phần còn lại vẫn ở điểm vận hành tốt	Không re-tune; hoặc thành phần bị bỏ kéo theo thay đổi ngầm
Add-one-in	Thành phần này tự nó có giá trị không	Baseline trần vẫn huấn luyện ổn định	Có tương tác mạnh: con số không cộng lại thành hệ thống đầy đủ
Sanity	Có rò rỉ, hay thành phần vô dụng	Metric đánh giá nhạy với thứ bị phá	Đọc nhầm train loss thay vì điểm đánh giá
Oracle / skyline	Trần đóng góp của khối này là bao nhiêu	Chân trị cho khối đó tồn tại và đúng định dạng	Oracle làm đổi phân phối đầu vào của khối sau, cho trần lạc quan giả
Theo quy mô	Kết luận có bền theo lượng dữ liệu không	Xu hướng đơn điệu giữa hai đầu	Chỉ đo hai điểm; giao nhau nằm giữa mà không thấy

I 1.4 Nhóm C — thống kê tối thiểu (Minimal Statistics)

A. Sàn nhiễu trước, cỡ mẫu sau

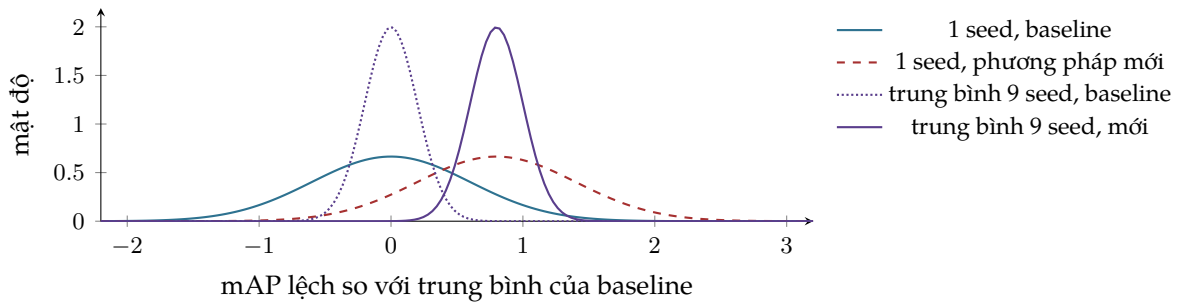
Quy tắc ba lần chạy của bạn đúng về nguyên tắc nhưng ba là *sàn*, không phải mục tiêu; số lần chạy cần thiết phụ thuộc vào tỉ số giữa hiệu ứng muốn phát hiện và độ lệch chuẩn giữa các seed. Thứ tự đúng vì thế là **đo sàn nhiễu (noise floor) trước, nói về cải tiến sau**: chạy *cùng một cấu*

hình nhiều lần chỉ đổi seed, lấy độ lệch chuẩn, và chỉ từ đó mới biết chênh lệch bao nhiêu là đáng nói. Một tổ chức không biết sản phẩm của mình sẽ dành nhiều tháng để đuổi theo nhiều.

Ví dụ tính tay được, dùng đúng những con số hay gặp trong detection. Giả sử độ lệch chuẩn giữa các seed là $\sigma = 0,6$ mAP và bạn muốn phát hiện một cải tiến thật $\Delta = 0,8$ mAP. Với kiểm định hai mẫu, xấp xỉ chuẩn cho cỡ mẫu mỗi nhóm

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\Delta^2} = \frac{2(1,96 + 0,84)^2 (0,6)^2}{(0,8)^2} \approx 8,8,$$

tức khoảng chín seed mỗi cấu hình, mười tám lần chạy cho một so sánh. Bây giờ đi ngược lại, với ba seed mỗi bên. Sai số chuẩn của hiệu số khi đó là $0,6\sqrt{2/3} \approx 0,49$. Hiệu số thật 0,8 chỉ nằm cách ngưỡng 1,96 đúng 0,33 độ lệch. Xác suất phát hiện được rơi vào khoảng 0,35 theo xấp xỉ chuẩn, và còn thấp hơn nếu tính bằng phân phối t với bậc tự do nhỏ. Nói cách khác, ở ba seed bạn bỏ sót khoảng hai phần ba số cải tiến có thật. Hình 1.2 cho thấy vì sao: hai phân phối một-lần-chạy chồng lên nhau gần hết, và chỉ khi lấy trung bình nhiều seed thì chúng mới tách ra.



Hình 1.2. Với $\sigma = 0,6$ mAP giữa các seed, một cải tiến thật 0,8 mAP nằm gần như hoàn toàn trong vùng chồng lấn của hai phân phối một-lần-chạy. Lấy trung bình chín seed thu hẹp độ lệch xuống $\sigma/3 = 0,2$ và tách hai phân phối ra. Đây là lý do hình học của công thức cỡ mẫu ở trên.

B. Hai cách rẻ hơn là tăng số seed

Có hai cách rẻ hơn là tăng số seed. Thứ nhất, **so sánh theo cặp** (*paired comparison*): hai mô hình được đánh giá trên đúng cùng một tập test, nên phần phương sai do độ khó của từng mẫu là chung và biến mất khi lấy hiệu số. Bootstrap theo cặp làm thế này: lấy mẫu lại các ảnh, rồi tính hiệu số metric trên mỗi lần lấy mẫu. Cách đó cho khoảng tin cậy chặt hơn hẳn so với việc so hai con số tổng, và không tốn thêm một lần huấn luyện nào. Thứ hai, báo cáo **cỡ hiệu ứng** (*effect size*) chứ không chỉ p-value: “có ý nghĩa thống kê” và “đáng để triển khai” là hai mệnh đề khác nhau, và với tập test đủ lớn thì mọi khác biệt đều có ý nghĩa thống kê. Khi phải so nhiều phương pháp trên nhiều tập dữ liệu thì các kiểm định phi tham số theo hạng là lựa chọn an toàn hơn kiểm định t [4].

C. So sánh bội và rò rỉ tập test

Vấn đề còn lại là **so sánh bội** (*multiple comparisons*), và nó nghiêm trọng hơn vẻ ngoài. Nếu bạn thử 20 biến thể đều vô dụng như nhau và báo cáo cái tốt nhất, thì cái tốt nhất đó là cực đại của 20 mẫu độc lập từ cùng phân phối nhiễu. Kỳ vọng của cực đại 20 mẫu chuẩn tắc vào khoảng 1,9; với $\sigma = 0,6$ mAP, đó là một “cải tiến” khoảng 1,1 mAP sinh ra hoàn toàn từ nhiễu. Con số này lớn hơn phần lớn cải tiến được công bố trong các bảng detection, và nó giải thích vì sao thứ tự thao tác — chọn trên val, báo cáo trên test đúng một lần — không phải hình thức. Cùng cơ chế đó vận hành chậm hơn nhưng chắc chắn hơn khi bạn dùng lại một tập test suốt nhiều tháng: mỗi quyết định thiết kế dựa trên điểm test là một lần rò rỉ nhỏ. Bằng chứng thực nghiệm ở đây tinh tế: khi dựng lại một tập test mới cho ImageNet và CIFAR-10 theo đúng quy trình gốc, mọi mô hình đều tụt điểm đáng kể, nhưng *thứ hạng giữa chúng gần như được bảo toàn* [5]. Diễn giải đúng là khoảng

cách tuyệt đối không đáng tin trong khi so sánh tương đối thì bền hơn ta lo — chứ không phải “rò rỉ thích ứng không tồn tại”.

CẠM BẤY

Hai dạng ô nhiễm đặc thù của deep learning mà kỷ luật SLAM không dạy bạn. *Một*, mô hình nền được pretrain trên dữ liệu web quy mô lớn có thể đã nhìn thấy chính ảnh trong tập test của bạn; khi đó “zero-shot” là một tuyên bố không kiểm chứng được nếu không dò trùng lặp. *Hai*, chính tập test có lỗi nhãn ở tỉ lệ không nhỏ trong các benchmark phổ biến, và ở vùng chênh lệch dưới một điểm thì thứ hạng có thể đảo khi sửa nhãn [6]. Cả hai đều không hiện ra dưới dạng phương sai giữa seed, nên chạy thêm seed không phát hiện được chúng.

D. Phi tất định trong deep learning

Cuối cùng là phần bạn đã sống qua nhưng ở dạng khác: nguồn phi tất định. Các nguồn chính trong deep learning:

- **Chọn thuật toán** — *autotune* của *cuDNN* chọn kernel tích chập khác nhau giữa các lần chạy tùy theo đo thời gian tức thời.
- **Cộng không có thứ tự** — các kernel dùng `atomicAdd` ở **lan truyền ngược** (*backpropagation*) cộng dồn theo thứ tự do lịch chạy quyết định; phép cộng dấu phẩy động không kết hợp, nên tổng đổi ở chữ số cuối. Đây đúng là cơ chế đã sinh ra chênh lệch một inlier trong hệ SLAM của bạn, chỉ khác là ở đây nó nằm trong thư viện chứ không nằm trong code bạn sửa được.
- **Độ chính xác rút gọn** — *TF32* và các đường *mixed precision* trên tensor core cho kết quả khác FP32 và khác nhau giữa các thế hệ GPU.
- **Thứ tự dữ liệu** — dataloader nhiều *worker* trả *batch* theo thứ tự hoàn thành, và mỗi worker có dòng số ngẫu nhiên riêng cho *augmentation*.

Bật chế độ tất định xử lý được vài nguồn trong số đó, phải trả bằng tốc độ, và *vẫn* không đảm bảo hai máy khác GPU cho cùng kết quả. Kết luận thực dụng — giống hết kết luận bạn đã rút ra cho harness của mình, chỉ khác là ở đây không có bug nào để sửa — là mục tiêu đúng phải chuyển từ tái lập từng chữ số sang tái lập phân phối: cùng seed, cùng máy thì khớp; khác máy thì trung bình và độ lệch chuẩn phải nằm trong khoảng tin cậy của nhau. Phần hạ tầng để làm được điều đó nằm ở D/22-tai-lap-va-mlops.

→ **cùng nguyên lý với D/22** — cùng lý do phải gắn mỗi lần chạy với một commit, một config và một phiên bản dữ liệu: cái không ghi lại được thì không so lại được

1.5 Nhóm D — phân tích lỗi (Error Analysis)

A. Một trăm lỗi gán tay

Đến đây bạn đã biết một cải tiến có thật hay không. Câu hỏi tiếp theo đắt hơn nhiều: nên đầu tư vào đâu. Không có công cụ tự động nào thay thế được việc **phân loại một trăm lỗi bằng tay**. Quy trình là lấy ngẫu nhiên một trăm mẫu bị sai, xem từng mẫu, dựng dần một taxonomy lỗi từ chính chúng, đếm tần suất mỗi loại, rồi ước lượng trần cải thiện nếu loại đó biến mất hoàn toàn. Cỡ mẫu một trăm không tùy tiện: với một loại chiếm 20%, sai số chuẩn của tỉ lệ là $\sqrt{0,2 \cdot 0,8/100} = 0,04$, tức là đủ để xếp hạng các loại lỗi lớn nhưng không đủ để phân biệt hai loại chênh nhau vài điểm

phần trăm. Điều đó là đủ, vì mục đích ở đây là chọn ra một hoặc hai loại đáng theo đuổi, chứ không phải xếp hạng chính xác.

B. Phân rã tự động và phân tích theo lát cắt

Phân rã tự động bổ sung chứ không thay thế bước thủ công. Với detection, các công cụ tách lỗi thành nhầm lớp, lệch vị trí, trùng lặp, bỏ sót và nền bị nhận nhầm; cách tính đúng đắn là đo phần mAP lấy lại được nếu sửa riêng từng loại, thay vì chỉ đếm số lượng [7], [8]. Giá trị của cách phân rã này là mỗi loại lỗi dẫn tới một cách sửa *khác nhau*:

- **Lệch vị trí** — hàm mất mát hồi quy hộp và độ phân giải đặc trưng.
- **Nhầm lớp** — dữ liệu và ranh giới định nghĩa lớp.
- **Bỏ sót** — quy tắc gán mẫu dương và ngưỡng điểm.
- **Trùng lặp** — hậu xử lý.

Bên cạnh đó là **phân tích theo lát cắt** (*slice analysis*): chia tập đánh giá theo các tiêu chí có nghĩa — kích thước vật thể, ánh sáng, camera, nhóm người — và đọc metric trên từng lát. Đây là nơi con số tổng nói dối nhiều nhất, vì một mô hình có thể giữ nguyên điểm tổng trong khi tụt sâu ở một lát chiếm 5% dữ liệu nhưng lại là 100% các trường hợp quan trọng. Với hệ thống robot, các lát tự nhiên là chuyển động nhanh, ánh mờ, bề mặt ít texture — và bạn đã biết rằng một con số trung bình trên toàn EuRoC che giấu hoàn toàn việc một chuỗi cụ thể sập.

C. Bốn ô hành động

Bước cuối khép vòng: mỗi loại lỗi phải được gán vào đúng một trong bốn ô hành động.

- **Sửa bằng dữ liệu** — lỗi tập trung ở một vùng phân phối thiếu mẫu, hoặc ở ranh giới định nghĩa lớp mà chính người gán nhãn cũng bất đồng.
- **Sửa bằng hàm mất mát (*loss function*)** — lỗi có dạng lệch hệ thống mà hàm mất mát hiện tại không phạt, ví dụ lệch vị trí hộp hoặc mất cân bằng lớp.
- **Sửa bằng kiến trúc** — lỗi do thiếu thông tin ở đầu vào của tầng quyết định: độ phân giải đặc trưng, trường tiếp nhận, phạm vi ngưỡng.
- **Sửa bằng ràng buộc hệ thống** — lọc theo thời gian, luật hậu kiểm, hoặc từ chối trả lời. Đây là ô mà một kỹ sư robot quen nhất, và cũng là ô rẻ nhất. Nó chỉ giấu lỗi đi chứ không sửa mô hình. Nhưng với các lỗi hiếm mà hậu quả lớn, giấu đúng cách lại chính là đáp án; xem E/25-do-tin-cay-va-van-hanh.

Việc gán này phải làm *trước* khi viết một dòng code, vì nó là chỗ duy nhất trong quy trình mà bạn còn ra quyết định bằng suy nghĩ thay vì bằng quán tính. Kinh nghiệm thực hành lặp lại ở nhiều nhóm — không phải một kết quả đo được — là ô thứ nhất thắng nhiều hơn người ta tưởng và ô thứ ba thắng ít hơn người ta tưởng.

1.6 Nhóm E — báo cáo và đọc ablation của người khác (Reporting)

↔ **tiền đề** B/06 — taxonomy lỗi dựng từ một trăm mẫu gán tay chính là đầu vào của vòng lỗi → dữ liệu ở chương chất lượng dữ liệu

A. Bảng ablation đạt chuẩn

Một bảng ablation đạt chuẩn phải mang theo bốn thứ mà phần lớn bảng đã công bố không có:

- **Số seed và độ lệch chuẩn** — thiếu hai con số này thì mỗi dòng chỉ là một mẫu duy nhất.
- **Ngân sách tính chỉnh của từng dòng** — mỗi dòng đã được thử bao nhiêu cấu hình.
- **Tập nào** — nói rõ ablation chạy trên validation chứ không phải trên test.
- **Thay đổi ngầm kéo theo** — bỏ một thành phần thường đổi luôn số tham số, thời gian huấn luyện và learning rate tối ưu; phải ghi rõ cái nào đã được re-tune.

Thiếu bốn thứ này thì bảng vẫn đọc được như một bảng, nhưng nó không còn là bằng chứng.

B. Ghi cả kết quả âm

Điểm thứ hai quan trọng hơn và ít được làm nhất: **ghi lại cả kết quả âm**. Một ý tưởng đã thử và không hiệu quả có giá trị gần bằng một ý tưởng hiệu quả, vì nó thu hẹp không gian tìm kiếm cho chính bạn sáu tháng sau và cho người tiếp theo. Bạn đã có sẵn thói quen này ở phía SLAM — vài kết luận NO-GO được ghi lại kèm điều kiện thí nghiệm — và nên giữ nguyên nó khi chuyển sang deep learning, với một điều chỉnh: kết quả âm chỉ có giá trị khi nó *ghi kèm phạm vi hiệu lực*. “Ý tưởng X không hiệu quả” là một câu mê tín; “ý tưởng X không cho cải tiến vượt sà nhiều $\pm 0,6$ mAP trên tập này, ở quy mô này, với ngân sách tune này, qua ba seed” là một kết quả. Mệnh đề thứ hai cho bạn biết điều kiện nào thay đổi thì đáng thử lại. Trong deep learning điều kiện thay đổi liên tục, nên một NO-GO không ghi điều kiện sẽ chặn bạn khỏi đúng cơ hội về sau.

C. Đọc ablation của người khác

Điểm thứ ba là mặt đọc, chính là chương 23 áp vào một bảng cụ thể: ablation chạy trên tập nào, bao nhiêu seed, baseline có được tune không, và thành phần bị ablate có kéo theo thay đổi ngầm nào không. Bốn câu hỏi đó loại được phần lớn bảng ablation rỗng, nhanh hơn mọi cách đọc khác.

CẦN NHỚ

Một kết luận âm được ghi lại *kèm điều kiện thí nghiệm* là một kết quả, không phải một thất bại. Nó là thứ duy nhất trong công việc thí nghiệm không bị mất giá theo thời gian, miễn là bạn ghi đủ điều kiện để biết khi nào nên thử lại. [T2]

1.7 Giới hạn và trường hợp hỏng

Kỷ luật trên có một chi phí thẳng thắn: chín seed nhân một thiết kế giai thừa là ngân sách rất ít nhóm trả nổi. Thoả hiệp thực dụng là phân tầng — sàng nhanh bằng một seed để loại ứng viên vô vọng, rồi chỉ chạy đầy đủ cho cấu hình sống sót. Nhưng vòng sàng đã tiêu một phần “ngân sách tin cậy”: cấu hình sống sót được chọn bởi nhiều cũng nhiều như bởi chất lượng, nên con số của vòng đầy đủ vẫn thiên lệch lên trên nếu dùng lại cùng tập dữ liệu. Cách sửa rẻ nhất là tách tập sàng khỏi tập xác nhận.

Chế độ hỏng thứ hai mang tính khái niệm. Ablation giả định các thành phần *tách rời được*, tức bỏ một thành phần không làm hệ thống chuyển sang chế độ vận hành khác. Giả định này đúng với phần lớn mạng feed-forward và sai với hệ có vòng phản hồi: trong một pipeline SLAM, chất lượng bản đồ chỉ phối việc chọn keyframe và ngược lại, nên bỏ một thành phần trong vòng đó cho ra một hệ thống khác chứ không phải “hệ thống cũ trừ thành phần”. Khi ấy “đóng góp của

thành phần” không còn là đại lượng xác định, và câu hỏi đúng đổi thành: hệ chuyển sang chế độ nào, và chế độ đó tốt hay xấu hơn ở đâu.

Chế độ hồng thứ ba là chính sự tự tin do thống kê mang lại: một khoảng tin cậy hẹp chỉ nói rằng bạn đã đo chính xác *trên phân phối của tập đánh giá*. Nếu tập đó không đại diện cho môi trường triển khai, mọi kỷ luật ở chương này chỉ giúp bạn sai một cách rất chính xác — địa hạt của B/08-dịch-chuyen-phan-phoi.

1.8 Thực hành

Repo và công cụ. [dbolya/tide](#) để phân rã lỗi detection; [voxel51/fiftyone](#) để cắt lát, xem lỗi và so hai mô hình trên cùng mẫu; [wandb/wandb](#) sweeps và [optuna](#) để quản lý nhiều lần chạy có kỷ luật; `scipy.stats` cộng một hàm bootstrap tự viết cho kiểm định theo cặp; [facebookresearch/hydra](#) multirun để một thay đổi luôn tương ứng một config diff. Nên đọc thêm phần ablation của những paper có kỷ luật thí nghiệm tốt — dòng ConvNeXt, dòng “bag of tricks”, các paper về công thức huấn luyện — vì chúng dạy phương pháp rõ hơn hẳn các paper kiến trúc. Danh sách công cụ này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; các nguyên tắc trong chương thì không.

Câu hỏi kiểm tra hiểu. Bạn bỏ một module và mAP giảm 1,5 — nêu bốn cách giải thích khác ngoài “module đó quan trọng”. Nếu phương sai giữa seed là $\pm 0,6$ mAP, cần bao nhiêu lần chạy để tin một cải tiến 0,8 mAP, và vì sao so sánh theo cặp lại rẻ hơn cách đó? Oracle experiment nào bạn nên chạy đầu tiên cho hệ thống hiện tại của mình, và nó chặn trên cái gì?

Mini project. Lấy một repo có sẵn, *viết trước* dự đoán cho năm ablation kèm con số bạn nghĩ sẽ thấy, chạy mỗi cái ba seed, rồi đối chiếu: chỗ nào đúng, chỗ nào sai, trực giác nào cần sửa. Cột “dự đoán” là phần quan trọng nhất, vì nó là thứ duy nhất biến kết quả thành thông tin về mô hình tư duy của bạn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
•• Vì sao ba loại câu hỏi thí nghiệm cần ba thiết kế?	Vì chúng hỏi những thứ khác nhau. “Có hoạt động không” cần kiểm soát chặt và nhiều seed; “vì sao hoạt động” cần ablation có re-tune; “hoạt động tới đâu” cần nhiều điểm dọc một trục, ít seed mỗi điểm.
••• Baseline	Phương pháp đối chứng mà kết quả mới phải vượt qua. Một baseline chưa được tinh chỉnh tử tế làm vô nghĩa mọi con số phía sau, và đó là cách tự lừa mình phổ biến nhất vì nó rẻ.
•• Thiết kế giai thừa	Chạy mọi tổ hợp bật/tắt của các thành phần thay vì đổi từng cái một. Với ba thành phần là 8 cấu hình so với 4, tức gấp đôi chi phí, đổi lại bạn thấy được toàn bộ tương tác bậc hai.
••• Leave-one-out	Bỏ từng thành phần khỏi hệ thống đầy đủ rồi đo lại. Nó trả lời “thành phần này còn đáng giữ không”, và chỉ đúng khi phần còn lại vẫn ở điểm vận hành tốt.
•• Add-one-in	Thêm từng thành phần vào baseline trần. Nó trả lời “thành phần này tự nó có giá trị không”; khi có tương tác, con số của nó tách khỏi leave-one-out, và khoảng cách ấy chính là lượng tương tác.
••• Vì sao ablation phải re-tune?	Vì bỏ một thành phần ổn định hoá thì learning rate tối ưu của phần còn lại nhỏ đi. Giữ nguyên learning rate cũ là đo “thành phần đó cộng một điểm vận hành đã sai”, và sai lệch luôn thổi phồng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Sanity ablation	Nhóm phép thử thô: thay đầu vào bằng nhiễu, xáo trộn nhãn, thay module bằng hằng số, cắt bỏ một modality. Rẻ nhất mà phá được nhiều giả thuyết nhất, nên chạy trước mọi ablation tinh vi.
●●● Xáo trộn nhãn mà điểm đánh giá không giảm — nghĩa là gì?	Hoặc có rò rỉ dữ liệu, hoặc thành phần đang xét vô dụng. Phân biệt với hiện tượng bình thường: train loss vẫn giảm khi xáo trộn nhãn là chuyện thường, chỉ có điểm trên tập đánh giá mới là bằng chứng.
●●● Oracle experiment	Thay một khối bằng chân trị rồi đo hệ thống tổng thể, để biết trần đóng góp của khối đó. Nếu detector hoàn hảo chỉ nâng hệ thống 2%, mọi công sức tối ưu detector đều bị chặn trên bởi 2%.
●●● Sàn nhiễu	Độ lệch chuẩn giữa các seed của <i>cùng một</i> cấu hình. Nó là đơn vị đo của mọi so sánh: chênh lệch nhỏ hơn nó thì chưa nói được gì, dù bạn lặp lại bao nhiêu lần.
●●● Cỡ mẫu tối thiểu	Số seed cần cho mỗi cấu hình, tính từ sàn nhiễu và độ lớn cải tiến muốn phát hiện: $n \approx 2(1,96 + 0,84)^2 \sigma^2 / \Delta^2$. Với $\sigma = 0,6$ và $\Delta = 0,8$ là khoảng chín seed mỗi bên.
●● So sánh theo cặp	Đánh giá hai mô hình trên đúng cùng tập test rồi lấy hiệu số trên từng mẫu. Phần phương sai do độ khó từng ảnh là chung nên biến mất, và bootstrap theo cặp cho khoảng tin cậy chặt hơn mà không tốn thêm lần huấn luyện nào.
●●● So sánh bội	Thử nhiều biến thể rồi báo cáo cái tốt nhất. Kỳ vọng của cực đại hai mươi lần rút đã cỡ 1,9 độ lệch chuẩn, nên với $\sigma = 0,6$ mAP thì “cải tiến” 1,1 điểm có thể là nhiễu thuần.
●●● Cải thiện 1,1 mAP sau khi thử hai mươi cấu hình — nghĩa là gì?	Nhiều khả năng là nhiễu. Cách phân biệt: chạy lại đúng cấu hình thắng đó với vài seed mới trên một tập giữ riêng; nếu cải tiến bốc hơi thì bạn vừa đo cỡ mẫu của cuộc tìm kiếm.
●● Rò rỉ tập test	Tập test mất dần tư cách trọng tài vì bị dùng lại hàng trăm lần, hoặc vì nó nằm sẵn trong dữ liệu pretraining. Nó không hiện ra dưới dạng phương sai giữa seed, nên chạy thêm seed không phát hiện được.
●● Vì sao chế độ tắt định vẫn không tái lập được?	Vì nguồn phi tắt định nằm trong thư viện và phần cứng: cuDNN chọn kernel khác nhau, atomics cộng không theo thứ tự, TF32 khác giữa các đời GPU. Mục tiêu đúng là tái lập phân phối, không phải từng chữ số.
●●● Hai máy khác GPU cho ATE lệch 3 cm trên cùng cấu hình — bug hay không?	So với sàn nhiễu đã đo trước đó. Nếu 3 cm nằm trong khoảng dao động giữa các seed thì đó là phi tắt định của thư viện, không phải bug. Nếu vượt hẳn, hãy nghi khác phiên bản thư viện, khác dữ liệu, hoặc một cấu hình chưa được ghi.
●● Vì sao công cụ tự động không thay được gán tay?	Vì công cụ chỉ đếm được các loại lỗi đã có tên sẵn. Gán tay là chỗ taxonomy mới sinh ra; sau đó công cụ mới đếm và ước lượng trần cải thiện cho từng loại.
●● Bốn ô hành động	Mỗi loại lỗi phải được gán vào đúng một ô: sửa bằng dữ liệu, bằng hàm mất mát, bằng kiến trúc, hoặc bằng ràng buộc hệ thống. Gán trước khi viết code, vì đó là chỗ duy nhất bạn còn quyết định bằng suy nghĩ.
●●● Kết quả âm	Ghi lại thứ đã thử và không hiệu quả, kèm tập nào, quy mô nào, mấy seed, sàn nhiễu bao nhiêu. Thiếu điều kiện thì nó thành mê tín và sẽ chặn bạn khỏi đúng cơ hội sáu tháng sau.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
dbolya/tide [7]	Phân rã lỗi detection theo loại, mỗi loại kèm phần mAP lấy lại được nếu sửa riêng nó — chính ý tưởng trần cải thiện theo loại lỗi	Kế thừa phân tích chẩn đoán của Hoiem [8]
voxel51/fiftyone	Cắt lát tập đánh giá và so hai mô hình trên cùng mẫu — hiện thực của phân tích theo lát cắt và so sánh theo cặp	Hạ tầng; bền hơn tên mô hình
Dòng ConvNeXt [9]	Cả paper là một chuỗi ablation recipe-so-với-kiến trúc có kiểm soát	Nên đọc phần ablation hơn phần kiến trúc
Reproducibility checklist (NeurIPS, ICLR)	Ép khai báo số seed, độ lệch chuẩn và ngân sách tính toán — đúng bốn yêu cầu của một bảng ablation đạt chuẩn	Thông lệ từ khoảng 2019
Benchmark harness EuRoC của chính bạn	Quy tắc ba lần chạy, chế độ lockstep để cô lập phi tắt định, các kết luận NO-GO có ghi điều kiện — nhóm C và nhóm E ở dạng thực địa	Ví dụ nội bộ; số liệu nằm trong ghi chú dự án

MINI PROJECT SLAM · Đo sần nhiễu của chính harness EuRoC của bạn · 1-2 ngày

Mục tiêu	biến câu “một lần chạy không nói lên điều gì” thành một con số cụ thể cho hệ SLAM của bạn. Từ con số đó suy ra số lần chạy tối thiểu cho mọi so sánh về sau.
Dữ liệu	hai chuỗi EuRoC có độ khó khác nhau (ví dụ một chuỗi dễ và V2_03), cấu hình mặc định, không đổi bất cứ tham số nào giữa các lần chạy.
Các bước	chạy <i>cùng một</i> cấu hình mười lần trên mỗi chuỗi; với mỗi lần ghi ATE bằng evo cùng tỉ lệ khung hình bấm được; lập bảng mười dòng, tính trung bình và độ lệch chuẩn σ riêng cho từng chuỗi; rồi áp công thức cỡ mẫu $n \approx 2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2 / \Delta^2$ cho hai mức Δ bạn quan tâm, chẳng hạn 1 cm và 3 cm.
Xong khi	bạn phát biểu được, kèm bảng số chống lưng, câu “trên chuỗi này, chênh lệch ATE nhỏ hơn X cm giữa hai cấu hình là nhiễu”, và nói được số lần chạy tối thiểu cần cho mỗi Δ . Kiểm tra chéo: σ của chuỗi khó phải lớn hơn hẳn σ của chuỗi dễ; nếu không, harness của bạn còn một nguồn phi tắt định chưa lộ.
Nối với	B/07 cho giao thức đánh giá và cách chọn chuỗi; D/22 cho việc gắn mỗi lần chạy với một commit và một config; E/25 sẽ dùng lại chính bảng số này khi đặt ngưỡng theo chi phí.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì con số bạn báo cáo là *cực đại* của hai mươi mẫu chứ không phải một mẫu, và kỳ vọng của cực đại tăng theo cỡ mẫu. Với 20 mẫu chuẩn tắc, kỳ vọng cực đại khoảng 1,9; nhân với $\sigma = 0,6$ ra $\approx 1,1$ điểm sinh ra hoàn toàn từ nhiễu.
- Với $\sigma = 0,6$ và cải tiến thật 0,8, cần khoảng chín seed mỗi bên. Ở ba seed, sai số chuẩn của hiệu số là $0,6\sqrt{2/3} \approx 0,49$, nên xác suất phát hiện chỉ khoảng 0,35 — bỏ sót chừng hai phần ba số cải tiến có thật. Ba là sần để phát hiện *sự bất định*, không phải mức đủ để *kết luận*.
- Bốn cách: chênh lệch nằm trong sần nhiễu; learning rate không được re-tune sau khi bỏ module; việc bỏ module kéo theo một thay đổi ngầm (số tham số, thời gian huấn luyện, đường dữ liệu); và module chỉ quan trọng ở *quy mô dữ liệu này*. Cách thứ hai luôn đẩy kết luận về cùng một hướng — thổi phồng — nên nó không tự triệt tiêu khi lặp lại nhiều lần.
- Không còn lý do nào: mọi công sức tối ưu detector bị chặn trên bởi 2%. Bạn biết con số ấy trước bằng một oracle experiment — thay detector bằng chân trị rồi đo hệ thống tổng thể — và đó là thí nghiệm nên chạy *đầu tiên* chứ không phải cuối cùng.
- Vì “X không hiệu quả” không cho biết điều kiện nào thay đổi thì đáng thử lại. Trong deep learning điều kiện đổi liên tục: quy mô dữ liệu, mô hình nền, ngân sách tính chính. Một NO-GO không ghi điều kiện sẽ chặn bạn khỏi đúng cơ hội về sau, trong khi không ghi gì thì ít nhất bạn còn thử lại.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn mua một cái cân trong nhà tắm. Bước lên ba lần trong cùng một phút, nó hiện ba số khác nhau, lệch nhau chừng ba lạng. Bây giờ bạn ăn kiêng một tuần rồi cân lại: nhẹ hơn hai lạng. Bạn có giảm cân thật không? Không ai trả lời được, vì hai lạng còn nhỏ hơn mức cái cân tự dao động. Câu hỏi phải hỏi trước là: cái cân này dao động bao nhiêu? Cân mười lần trong cùng một buổi sáng, ghi cả mười số lại, rồi mới biết chênh lệch bao nhiêu mới đáng nói.

Từ đó ra hai điều. Thứ nhất, muốn phát hiện một thay đổi nhỏ hơn mức dao động, bạn phải cân nhiều lần hơn — và số lần ấy tính ra được chứ không phải chọn theo thói quen. Thứ hai, nếu bạn thử hai mươi thực đơn rồi chỉ kể lại cái cho số đẹp nhất, thì phần lớn cái “đẹp nhất” ấy là do cái cân chứ không do thực đơn.

Cái giá phải trả: cân mười lần thì tốn gấp mười lần công. Và bạn phải chịu khó ghi lại cả những tuần không giảm được lạng nào, kèm điều kiện đầy đủ — ăn gì, cân lúc nào, cân nào — vì chính những đồng đó ngăn bạn thử lại đúng thực đơn ấy vào tháng sau.

Chỗ không rút gọn được: phần tính toán. Rút gọn cả chương thành “chạy nhiều lần cho chắc” nghe rất hợp lý nhưng làm mất đúng chỗ có giá trị: số lần chạy không phải một thói quen mà là kết quả của một phép tính, suy ra từ mức dao động bạn đã đo và từ độ lớn cải thiện bạn muốn phát hiện. Ba lần là thói quen; chín lần là một phép tính — và khác biệt giữa hai con số đó là bạn nói được mình đang chấp nhận bỏ sót bao nhiêu.

Một câu: Trước khi hỏi câu hình nào tốt hơn, hãy hỏi cái cân của bạn dao động bao nhiêu.

1.9 Kết nối

B/07-chia-du-lieu-va-danh-gia là tiền đề trực tiếp: cách chia dữ liệu sai thì mọi thống kê ở đây chỉ đo rất chính xác một đại lượng vô nghĩa. Phần đọc *ý nghĩa* của từng con số — precision, recall, F1, đường ROC và PR, hiệu chỉnh, và bảng chẩn đoán ngược từ triệu chứng về nguyên nhân — cũng nằm ở B/07-chia-du-lieu-va-danh-gia. E/23-doc-paper là mặt đối xứng: tiêu chí bạn dùng để nghi ngờ ablation của người khác chính là tiêu chí bạn phải tự thoả. D/22-tai-lap-va-mlops là hạ tầng khiến kỷ luật này khả thi — một thay đổi bằng một config diff, mỗi lần chạy gắn với một commit và một phiên bản dữ liệu. B/08-dich-chuyen-phan-phoi là giới hạn ngoài: nó cho thấy khi nào một kết luận đúng trên tập đánh giá vẫn không chuyển được sang thực tế. Cuối cùng, E/26-lo-trinh-hoc cho biết chương này nằm ở đâu trong từng lộ trình theo mục tiêu — nó là chương trung tâm của mọi lộ trình có chữ “đánh giá” trong mục tiêu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao sai lệch do không re-tune sau khi bỏ một thành phần lại luôn cùng một chiều, và vì sao chạy thêm seed không sửa được sai lệch đó?
2. Leave-one-out và add-one-in cho hai con số khác nhau cho cùng một thành phần. Điều đó nói lên gì về hệ thống, và bạn báo cáo con số nào?
3. Với $\sigma = 0,6$ và 20 biến thể vô dụng như nhau, “cải tiến” tốt nhất bạn kỳ vọng thấy là bao nhiêu? Con số đó so với các cải tiến thường được công bố thì thế nào?
4. Bật chế độ tắt định của framework rồi mà hai máy vẫn cho kết quả khác nhau: ba nguyên nhân là gì, và vì sao mục tiêu “tái lập từng chữ số” là mục tiêu sai?
5. Một oracle experiment cho trần lạc quan giả trong trường hợp nào — cụ thể, chuyện gì xảy ra với phân phối đầu vào của khối đứng ngay sau oracle?
6. Vì sao một kết luận NO-GO không ghi điều kiện thí nghiệm lại có hại hơn là không ghi gì?
7. Ablation giả định điều gì về cấu trúc hệ thống, và giả định đó hỏng thế nào với một hệ có vòng phản hồi?

Nguồn

- [1] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [2] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [3] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [4] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [5] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [6] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [7] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [8] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [9] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.